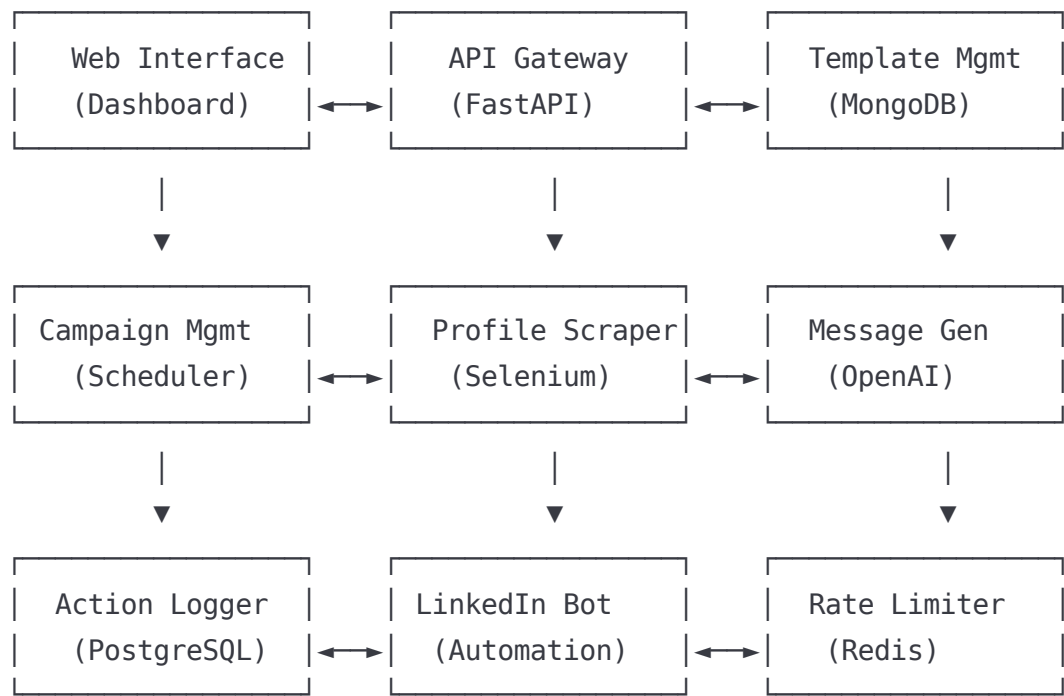


LinkedIn Outreach Automation Agent

Architecture Overview

Core Components



Modular Architecture

1. Core Modules

A. Profile Scraper Module

python

```
class ProfileScraper:
    """
    Handles LinkedIn profile data extraction
    """
    def __init__(self, driver_config: dict):
        self.driver = self._initialize_driver(driver_config)

    def extract_profile_data(self, profile_url: str) -> ProfileData:
        """Extract key profile information"""

    def extract_from_search(self, search_url: str) -> List[ProfileData]:
        """Extract profiles from search results"""

    def _initialize_driver(self, config: dict) -> webdriver.Chrome:
        """Initialize Selenium driver with stealth settings"""
```

B. Message Generation Module

python

```
class MessageGenerator:
    """
    AI-powered message personalization using OpenAI
    """
    def __init__(self, openai_api_key: str):
        self.client = OpenAI(api_key=openai_api_key)

    def generate_connection_request(self, profile: ProfileData, template: str) -> str:
        """Generate personalized connection request"""

    def generate_direct_message(self, profile: ProfileData, template: str) -> str:
        """Generate personalized direct message"""

    def generate_thank_you_message(self, profile: ProfileData, template: str) -> str:
        """Generate thank you message for accepted connections"""
```

C. LinkedIn Automation Module

python

```
class LinkedInBot:
    """
    Handles LinkedIn UI interactions
    """
    def __init__(self, driver, rate_limiter):
        self.driver = driver
        self.rate_limiter = rate_limiter

    def send_connection_request(self, profile_url: str, message: str) -> bool:
        """Send connection request with personalized note"""

    def send_direct_message(self, profile_url: str, message: str) -> bool:
        """Send direct message to profile"""

    def check_accepted_connections(self) -> List[str]:
        """Check for newly accepted connection requests"""

    def send_thank_you_message(self, profile_url: str, message: str) -> bool:
        """Send thank you message to new connections"""
```

2. Data Models

python

```
from dataclasses import dataclass
from typing import Optional, List
from datetime import datetime
```

@dataclass

```
class ProfileData:
    url: str
    name: str
    headline: str
    location: str
    experience: List[str]
    education: List[str]
    skills: List[str]
    mutual_connections: int
    scraped_at: datetime
```

@dataclass

```
class Campaign:
    id: str
    name: str
    input_type: str # 'search_url' or 'profile_list'
    input_data: str
    action_type: str # 'connection' or 'message'
    connection_template: Optional[str]
    message_template: Optional[str]
    thank_you_template: str
    daily_limit: int
    created_at: datetime
    status: str # 'active', 'paused', 'completed'
```

@dataclass

```
class OutreachAction:
    id: str
    campaign_id: str
    profile_url: str
    action_type: str
    message_sent: str
    status: str # 'sent', 'accepted', 'declined', 'pending'
    sent_at: datetime
    response_at: Optional[datetime]
```

3. Template Management System

python

```
class TemplateManager:
    """
    Manages global templates and system prompts
    """
    def __init__(self, db_connection):
        self.db = db_connection

    def get_template(self, template_type: str, template_name: str) -> str:
        """Retrieve template by type and name"""

    def update_template(self, template_type: str, template_name: str, content: str):
        """Update template without code changes"""

    def get_system_prompt(self, prompt_type: str) -> str:
        """Get system prompt for AI message generation"""
```

API Usage Plan

OpenAI Integration


```

class AIMessageService:
    def __init__(self):
        self.system_prompts = {
            'connection_request': """
            You are a professional LinkedIn outreach expert. Generate a personalized
            connection request note (max 200 characters) that is:
            - Professional and warm
            - References specific details from their profile
            - Clearly states the reason for connecting
            - Avoids generic templates
            """,

            'direct_message': """
            You are a professional LinkedIn outreach expert. Generate a personalized
            direct message that is:
            - Professional and conversational
            - References their background/experience
            - Clearly states your purpose
            - Includes a clear call-to-action
            - Keeps it concise (under 150 words)
            """,

            'thank_you': """
            Generate a brief, professional thank-you message for someone who
            accepted your LinkedIn connection request. Should be:
            - Grateful and professional
            - Reference their background
            - Suggest next steps if appropriate
            - Keep it under 100 words
            """
        }

    def generate_personalized_message(self, message_type: str, profile: ProfileData,
                                     template: str) -> str:
        user_prompt = f"""
        Template: {template}

        Profile Information:
        - Name: {profile.name}
        - Headline: {profile.headline}
        - Location: {profile.location}
        - Recent Experience: {profile.experience[:2]}
        - Skills: {profile.skills[:5]}

        Generate a personalized message based on this information.
        """

```

```
response = self.openai_client.chat.completions.create(
    model="gpt-4",
    messages=[
        {"role": "system", "content": self.system_prompts[message_type]},
        {"role": "user", "content": user_prompt}
    ],
    max_tokens=300,
    temperature=0.7
)

return response.choices[0].message.content
```

Step-by-Step Workflow

Phase 1: Campaign Setup

1. Input Processing

- Accept LinkedIn search URL or profile list
- Validate inputs and extract profile URLs
- Create campaign record in database

2. Profile Data Collection

- Use Selenium to scrape profile information
- Implement random delays (2-5 seconds between requests)
- Store profile data in database
- Handle rate limiting and errors gracefully

Phase 2: Message Generation

1. Template Loading

- Load appropriate global templates from database
- Load system prompts for AI generation

2. AI-Powered Personalization

- For each profile, generate personalized message
- Use OpenAI API with profile-specific context
- Validate message length and content

Phase 3: Automated Outreach

1. Connection Requests

python

```
def send_connection_request(self, profile_url: str, message: str):
    # Navigate to profile
    self.driver.get(profile_url)
    time.sleep(random.uniform(3, 7)) # Human-like delay

    # Click connect button
    connect_btn = self.wait.until(
        EC.element_to_be_clickable((By.XPATH, "//button[contains(text(), 'Connect')]"))
    )
    connect_btn.click()

    # Add personalized note
    note_area = self.wait.until(
        EC.presence_of_element_located((By.ID, "custom-message"))
    )
    note_area.send_keys(message)

    # Send request
    send_btn = self.driver.find_element(By.XPATH, "//button[contains(text(), 'Send')]")
    send_btn.click()

    # Log action
    self.logger.log_action('connection_request', profile_url, message)
```

2. Direct Messages

python

```
def send_direct_message(self, profile_url: str, message: str):
    # Navigate to profile
    self.driver.get(profile_url)
    time.sleep(random.uniform(3, 7))

    # Click message button
    message_btn = self.wait.until(
        EC.element_to_be_clickable((By.XPATH, "//button[contains(text(), 'Message')]"))
    )
    message_btn.click()

    # Type message
    message_box = self.wait.until(
        EC.presence_of_element_located((By.CSS_SELECTOR, "[data-artdeco-is-focused=]"))
    )
    message_box.send_keys(message)

    # Send message
    send_btn = self.driver.find_element(By.XPATH, "//button[contains(text(), 'Send')]")
    send_btn.click()

    # Log action
    self.logger.log_action('direct_message', profile_url, message)
```



Phase 4: Follow-up Management

1. Connection Monitoring

- Periodically check for accepted connections
- Compare current connections with previous state
- Identify newly accepted requests

2. Automated Thank-you Messages

- Generate personalized thank-you messages
- Send to newly accepted connections
- Log all follow-up actions

Rate Limiting & Safety Features

1. Intelligent Rate Limiting

python

```
class RateLimiter:
    def __init__(self, redis_client):
        self.redis = redis_client
        self.limits = {
            'connection_requests': {'daily': 20, 'weekly': 100},
            'messages': {'daily': 50, 'weekly': 200},
            'profile_views': {'daily': 100, 'weekly': 500}
        }

    def check_limit(self, action_type: str, user_id: str) -> bool:
        """Check if action is within rate limits"""

    def increment_counter(self, action_type: str, user_id: str):
        """Increment action counter"""
```

2. Human-like Behavior Patterns

python

```
class BehaviorMimicry:
    @staticmethod
    def random_delay(min_seconds: int = 2, max_seconds: int = 8):
        """Add random delays to mimic human behavior"""
        time.sleep(random.uniform(min_seconds, max_seconds))

    @staticmethod
    def scroll_behavior(driver):
        """Simulate human scrolling patterns"""
        driver.execute_script("window.scrollTo(0, document.body.scrollHeight/2);")
        time.sleep(random.uniform(1, 3))

    @staticmethod
    def typing_pattern(element, text: str):
        """Simulate human typing patterns"""
        for char in text:
            element.send_keys(char)
            time.sleep(random.uniform(0.05, 0.2))
```

Ethical Considerations & LinkedIn Compliance

1. Rate Limiting

- Daily connection requests: 20 maximum
- Daily messages: 50 maximum

- Weekly profile views: 500 maximum
- Random delays between actions (3-8 seconds)

2. Content Guidelines

- Personalized messages only (no generic spam)
- Professional tone maintenance
- Respect user privacy and LinkedIn ToS
- No aggressive follow-up sequences

3. Detection Avoidance

python

```
class StealthMode:
    def setup_driver(self):
        options = webdriver.ChromeOptions()
        options.add_argument("--disable-blink-features=AutomationControlled")
        options.add_experimental_option("excludeSwitches", ["enable-automation"])
        options.add_experimental_option('useAutomationExtension', False)

        driver = webdriver.Chrome(options=options)
        driver.execute_script("Object.defineProperty(navigator, 'webdriver', {get: () :
        return driver
```

Configuration Management

1. Template System

json

```
{
  "connection_templates": {
    "default": "Hi {name}, I came across your profile and was impressed by your work in {field}. I work with companies like yours and I'm interested in {field}."
    "sales": "Hi {name}, I noticed your expertise in {field}. I work with companies like yours and I'm interested in {field}."
    "recruiting": "Hi {name}, your background in {field} caught my attention. I'm working on a project that requires someone with your skills."
  },
  "message_templates": {
    "default": "Hi {name},\n\nI hope this message finds you well. I came across your profile and was impressed by your work in {field}. I work with companies like yours and I'm interested in {field}."
  },
  "thank_you_templates": {
    "default": "Hi {name},\n\nThank you for connecting! I'm excited to be part of your network. I'll be sure to keep you updated on any new opportunities that arise."
  }
}
```

2. System Configuration

python

```
class Config:
    # API Keys
    OPENAI_API_KEY = os.getenv('OPENAI_API_KEY')

    # Database
    DATABASE_URL = os.getenv('DATABASE_URL')
    REDIS_URL = os.getenv('REDIS_URL')

    # Rate Limits
    DAILY_CONNECTION_LIMIT = 20
    DAILY_MESSAGE_LIMIT = 50

    # Automation Settings
    MIN_DELAY = 3
    MAX_DELAY = 8
    SCROLL_DELAY = 2

    # LinkedIn Settings
    LINKEDIN_EMAIL = os.getenv('LINKEDIN_EMAIL')
    LINKEDIN_PASSWORD = os.getenv('LINKEDIN_PASSWORD')
```

Monitoring & Logging

1. Comprehensive Logging

python

```
class OutreachLogger:
    def __init__(self, db_connection):
        self.db = db_connection

    def log_action(self, action_type: str, profile_url: str, message: str,
                  status: str = 'sent'):
        """Log all outreach actions"""
        log_entry = {
            'timestamp': datetime.utcnow(),
            'action_type': action_type,
            'profile_url': profile_url,
            'message': message,
            'status': status,
            'user_agent': self.get_user_agent(),
            'ip_address': self.get_ip_address()
        }
        self.db.outreach_logs.insert_one(log_entry)

    def log_error(self, error_type: str, error_message: str, profile_url: str):
        """Log errors and exceptions"""

    def generate_report(self, campaign_id: str) -> dict:
        """Generate campaign performance report"""
```

2. Dashboard Metrics

- Connection request sent/accepted rates
- Message response rates
- Daily/weekly activity summaries
- Error rates and types
- Profile scraping success rates

Deployment Architecture

1. Docker Composition

dockerfile

Dockerfile

FROM python:3.9-slim

RUN apt-get update && apt-get install -y \
chromium \
chromium-driver \
&& rm -rf /var/lib/apt/lists/*

WORKDIR /app

COPY requirements.txt .

RUN pip install -r requirements.txt

COPY . .

CMD ["python", "main.py"]

2. Environment Setup

yaml

```
# docker-compose.yml
version: '3.8'
services:
  linkedin-bot:
    build: .
    environment:
      - OPENAI_API_KEY=${OPENAI_API_KEY}
      - DATABASE_URL=${DATABASE_URL}
      - REDIS_URL=${REDIS_URL}
    depends_on:
      - postgres
      - redis
      - mongodb

  postgres:
    image: postgres:13
    environment:
      POSTGRES_DB: linkedin_outreach
      POSTGRES_USER: ${DB_USER}
      POSTGRES_PASSWORD: ${DB_PASSWORD}

  redis:
    image: redis:6-alpine

  mongodb:
    image: mongo:5
```

Usage Example

python

Initialize the outreach agent

```
agent = LinkedInOutreachAgent(  
    openai_api_key="your-openai-key",  
    database_url="postgresql://...",  
    redis_url="redis://..."  
)
```

Create a campaign

```
campaign = agent.create_campaign(  
    name="Software Developer Outreach",  
    input_type="search_url",  
    input_data="https://linkedin.com/search/results/people/?keywords=software%20develo",  
    action_type="connection",  
    connection_template="Hi {name}, I noticed your expertise in {field}...",  
    daily_limit=15  
)
```

Start the campaign

```
agent.start_campaign(campaign.id)
```

Monitor progress

```
report = agent.get_campaign_report(campaign.id)  
print(f"Sent: {report['sent']}, Accepted: {report['accepted']}")
```



This architecture provides a robust, scalable, and ethical LinkedIn outreach automation system that respects platform guidelines while maximizing personalization and effectiveness.